

2.K 최근접 이웃 회귀(KNN Regression) 알고리즘

학생점수데이터셋 <https://www.kaggle.com/yasserh/student-marks-dataset>
(<https://www.kaggle.com/yasserh/student-marks-dataset>)

데이터 전처리

In [1]:

```
#데이터 저장
```

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('Student_Marks.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   number_courses  100 non-null   int64
1   time_study      100 non-null   float64
2   Marks           100 non-null   float64
dtypes: float64(2), int64(1)
memory usage: 2.5 KB
```

In [2]:

```
#상관도 출력
```

```
df.corr()
```

Out[2]:

	number_courses	time_study	Marks
number_courses	1.000000	0.204844	0.417335
time_study	0.204844	1.000000	0.942254
Marks	0.417335	0.942254	1.000000

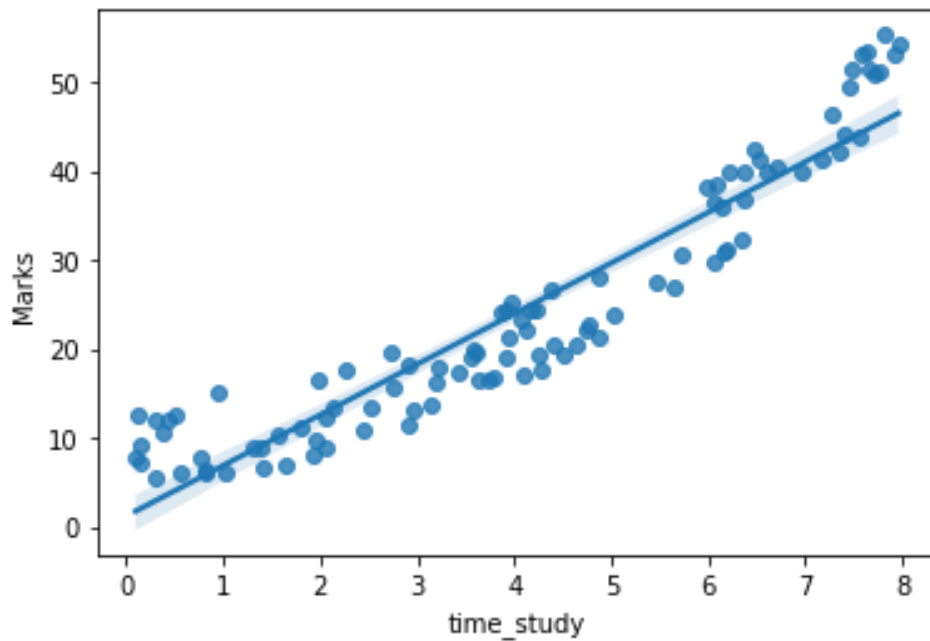
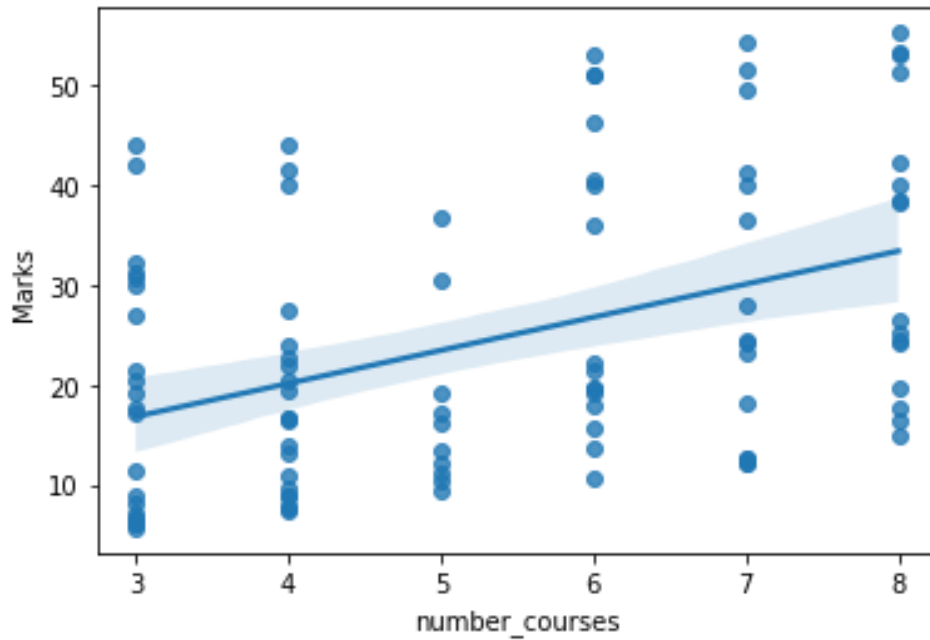
```
In [3]:
```

```
#그래프로 그리기
```

```
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
sns.regplot(x='number_courses', y='Marks', data=df)  
plt.show()
```

```
sns.regplot(x='time_study', y='Marks', data=df)  
plt.show()
```



In [4]:

```
#학습 데이터와 테스트 데이터 분리
from sklearn.model_selection import train_test_split
x = df.iloc[:, :-1]
y = df['Marks']

x_train, x_test, y_train, y_test=train_test_split(x,y,train_size=0.8,random_state=0)
```

모델 생성 및 학습

In [5]:

```
#머신 러닝 학습
from sklearn.neighbors import KNeighborsRegressor

KN_model = KNeighborsRegressor()
KN_model.fit(x_train, y_train)
```

Out[5]:

KNeighborsRegressor()

In [6]:

```
#예측하기
KN_prd = KN_model.predict(x_test)
pd.DataFrame({'예측값':KN_prd, '실제값':y_test, '예측-실제':KN_prd-y_test})
```

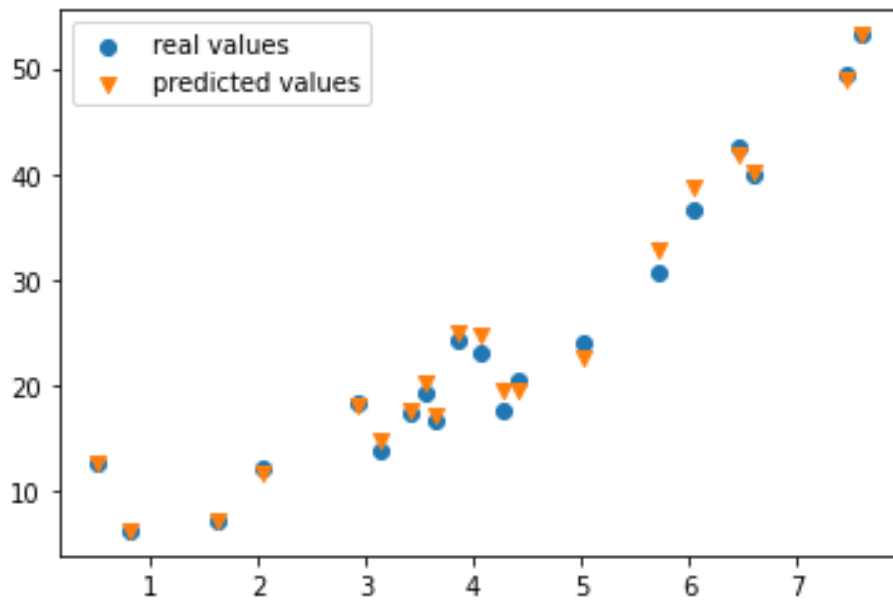
Out[6]:

	예측값	실제값	예측-실제
26	12.4620	12.647	-0.1850
86	24.8144	23.149	1.6654
2	14.6756	13.811	0.8646
55	18.1174	18.238	-0.1206
75	6.1638	6.217	-0.0532
93	41.8852	42.426	-0.5408
16	32.7114	30.548	2.1634
73	7.2098	7.014	0.1958
54	53.1810	53.158	0.0230
95	20.1534	19.128	1.0254
53	38.8178	36.653	2.1648
92	22.6376	23.916	-1.2784
78	49.0116	49.544	-0.5324
13	19.5232	17.672	1.8512
7	17.5978	17.264	0.3338
30	24.9656	24.172	0.7936
22	11.6804	12.209	-0.5286
24	17.0000	16.517	0.4830
33	40.2196	39.965	0.2546
8	19.5098	20.348	-0.8382

In [7]:

```
#그래프로 살펴보기
```

```
plt.scatter(x_test['time_study'],y_test,label='real values')
plt.scatter(x_test['time_study'],KN_prd,label='predicted values',marker='v')
plt.legend()
plt.show()
```



모델 평가

In [8]:

```
#평가하기
```

```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

```
mse = mean_squared_error(y_test, KN_prd)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, KN_prd)
score = KN_model.score(x_test, y_test)
```

```
print('mse =',mse)
print('rmse =', rmse)
print('r2 score =',r2)
print('score =',score)
```

```
mse = 1.0844076739999999
rmse = 1.041348968405884
r2 score = 0.9937819644602224
score = 0.9937819644602224
```

In [9]:

```
#선형 회귀 모델 생성, 학습, 예측, 평가
from sklearn.linear_model import LinearRegression

LN_model=LinearRegression()
LN_model.fit(x_train, y_train)
LN_prd = LN_model.predict(x_test)

mse = mean_squared_error(y_test, LN_prd)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, LN_prd)
score = LN_model.score(x_test, y_test)

print('mse =',mse)
print('rmse =', rmse)
print('r2 score =',r2)
print('score =',score)
```

```
mse = 10.823984498942073
rmse = 3.2899824466009044
r2 score = 0.9379348542894724
score = 0.9379348542894724
```


In [10]:

```
# 모듈 import
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

# 데이터를 가져오기
df = pd.read_csv('Student_Marks.csv')
df.info()

#상관도 분석
df.corr()
sns.regplot(x='number_courses', y='Marks', data=df)
plt.show()
sns.regplot(x='time_study', y='Marks', data=df)
plt.show()

#학습 데이터와 테스트 데이터 분리
x = df.iloc[:, :-1]
y = df['Marks']
x_train, x_test, y_train, y_test=train_test_split(x,y,
                                                    train_size=0.8,
                                                    random_state=0)

#K 최근접 이웃 회귀 모델 생성 학습 예측
KN_model = KNeighborsRegressor()
KN_model.fit(x_train, y_train)
KN_prd = KN_model.predict(x_test)

#K 최근접 이웃 회귀 모델예측 결과 확인하기
pd.DataFrame({'예측값':KN_prd,
              '실제값':y_test,
              '예측-실제':KN_prd-y_test})
plt.scatter(x_test['time_study'],y_test,c='r',label='real values')
plt.scatter(x_test['time_study'],KN_prd,label='predicted values')
plt.legend()
plt.show()

#K 최근접 이웃 회귀 모델 평가하기
mse = mean_squared_error(y_test, KN_prd)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, KN_prd)
score = KN_model.score(x_test, y_test)

print('mse =',mse)
```

```
print('rmse =', rmse)
print('r2 score =', r2)
print('score =', score)

#선형 회귀 모델 학습, 예측, 평가
LN_model=LinearRegression()
LN_model.fit(x_train, y_train)
LN_prd = LN_model.predict(x_test)

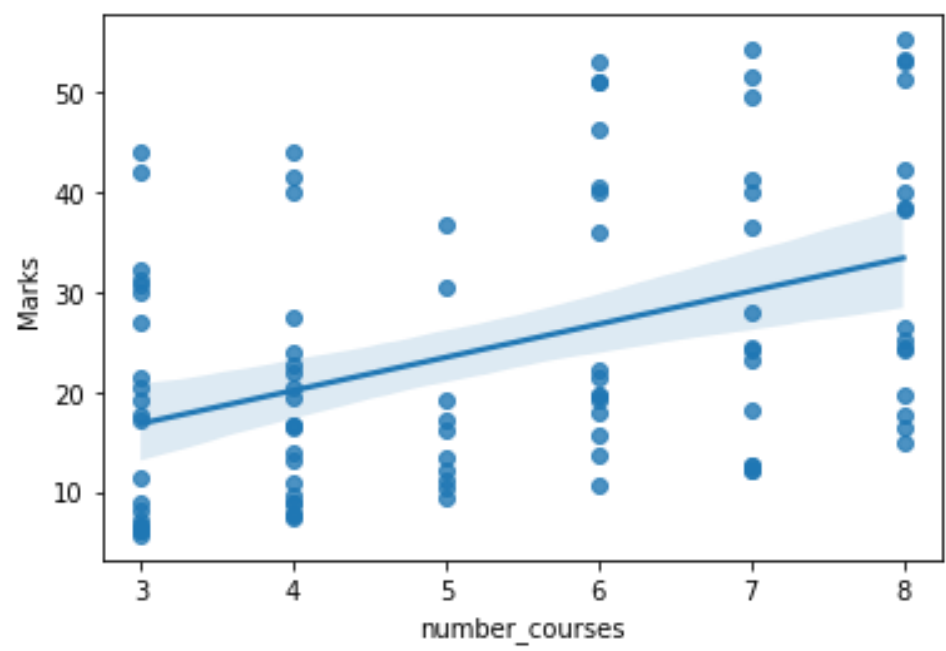
mse = mean_squared_error(y_test, LN_prd)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, LN_prd)
score = LN_model.score(x_test, y_test)
```

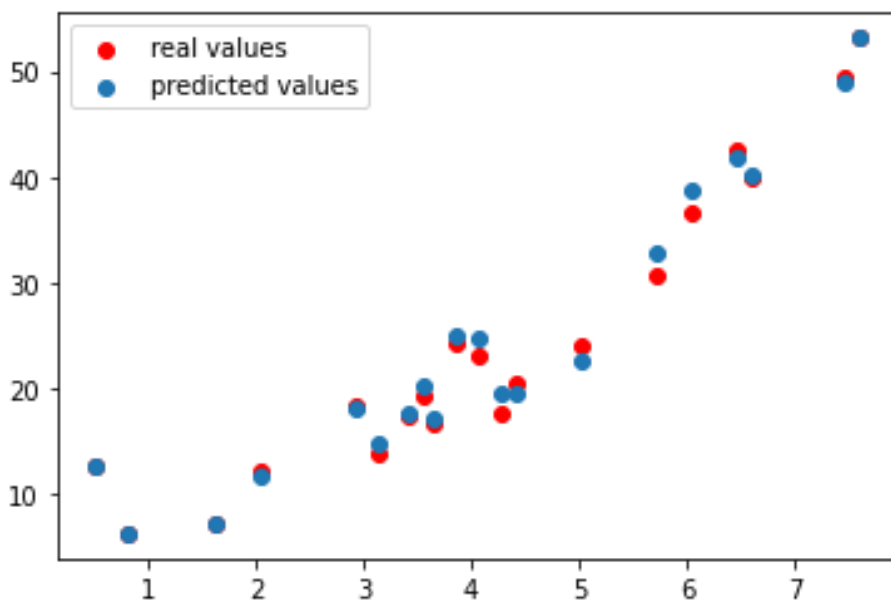
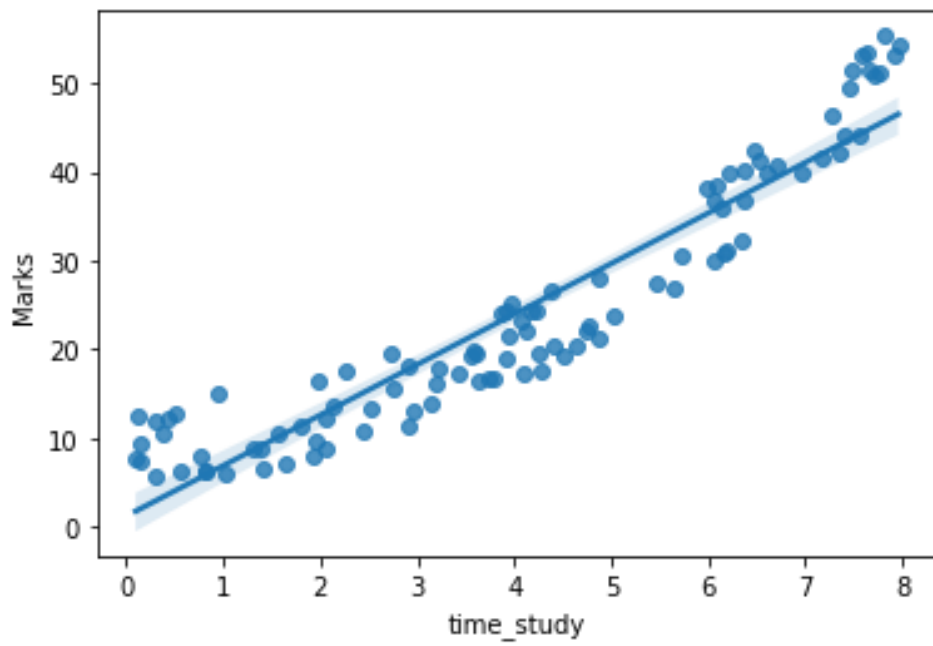
```
print('mse =',mse)
print('rmse =', rmse)
print('r2 score =',r2)
class pandas.core.frame.DataFrame>
```

RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	number_courses	100 non-null	int64
1	time_study	100 non-null	float64
2	Marks	100 non-null	float64

dtypes: float64(2), int64(1)
memory usage: 2.5 KB





$mse = 1.084407673999999$
 $rmse = 1.041348968405884$
 $r^2 \text{ score} = 0.9937819644602224$
 $\text{score} = 0.9937819644602224$
 $mse = 10.823984498942073$
 $rmse = 3.2899824466009044$
 $r^2 \text{ score} = 0.9379348542894724$
 $\text{score} = 0.9379348542894724$

In [11]:

#머신 러닝 학습

```
from sklearn.linear_model import LinearRegression
```

```
area_x = np.array( [[2.1], [10], [3], [1],  
                   [3.5], [5], [8]] , dtype=float)  
yield_y = np.array( [64.9, 292.6, 85.9, 30.92,  
                    110.5, 163.4, 230.1] , dtype=float)
```

#학습 모델 객체 생성

```
model1 = LinearRegression()  
model1.fit(area_x, yield_y)  
prd1 = model1.predict([[7], [12]])
```

```
model2 = KNeighborsRegressor()  
model2.fit(area_x, yield_y)  
prd2 = model2.predict([[7], [12]])
```

```
plt.scatter(area_x, yield_y, label='Train Data')  
plt.scatter([7, 12], prd1, label='Predicted Data', marker='v')  
plt.legend()  
plt.show()
```

```
plt.scatter(area_x, yield_y, label='Train Data')  
plt.scatter([7, 12], prd2, label='Predicted Data', marker='v')  
plt.legend()  
plt.show()
```

